Versionierung/Versionverwaltung

Aufgabe 1)
In der Veranstaltung wurden drei Versionen vorgestellt, die bei der (klassischen Form der) Weiterentwicklung eines Softwareprodukts, welches bereits produktiv ist, mindestens vorhanden sein müssen. Geben Sie die drei Versionen an und erläutern Sie diese bezüglich der Arbeitsschritte (1-2 Sätze je Version).

- Entwicklungsversion: In dieser Version wird in einzelnen Releases das Softwareprodukt erstellt und weiterentwickelt. Innerhalb eines Releases werden die Phasen des Wasserfallmodells durchlaufen.
- Releaseversion: In dieser Version werden vor einem Release diejenigen Teile der Entwicklungsversion zusammengestellt, die produktiv werden sollen. Diese Version wird kurz vor dem Release getestet.
- Produktivversion: Diese Version ist die ausgetestete Releaseversion, die produktiv geschaltet wird, dort nochmals getestet wird und dann zwecks Nutzung dem Kunden zur Verfügung steht.

Aufgabe 2)
Erläutern Sie kurz (2-3 Sätze): Was versteht man im Zusammenhang mit Versionsverwaltung unter Optimistic Locking / Pessimistic Locking?

→ Beim Optimistic Locking ist das Auschecken einer Datei jederzeit möglich, wobei Konflikte durch parallele Änderungen erst beim späteren Einchecken abgeglichen werden. Beim Pessimistic Locking hingegen wird eine Datei beim Auschecken exklusiv gesperrt, sodass kein anderer Entwickler gleichzeitig daran arbeiten kann. Beide Konzepte zur Konfliktvermeidung existieren analog auch bei Datenbankzugriffen.

Aufgabe 3)
Erläutern Sie kurz (2-3 Sätze): Was versteht man im Zusammenhang mit Versionsverwaltung unter Einchecken mit einem Konflikt?

→ Ein Konflikt beim Einchecken entsteht, wenn Entwickler dieselbe Datei parallel bearbeiten und einer seine Version später einreichen möchte. Zur Lösung muss dieser Entwickler zunächst die bereits im Repository gespeicherten Änderungen in seine lokale Datei integrieren (Merge). Erst nach dieser erfolgreichen Zusammenführung beider Bearbeitungsstände kann die finale, kombinierte Version eingecheckt werden.

Aufgabe 4)
Erläutern Sie kurz (2-3 Sätze): Was versteht man im Zusammenhang mit Versionsverwaltung unter Checkout?

→ Beim Checkout erzeugt ein Entwickler nach dem sogenannten Sandbox-Prinzip zunächst eine lokale Arbeitskopie des Repository-Inhalts, auf die sich alle Programmänderungen vorerst beschränken. Ob dieser Vorgang jederzeit möglich ist, hängt von der Locking-Strategie ab: Beim Optimistic Locking ist das Auschecken immer erlaubt, während es beim Pessimistic Locking nur möglich ist, wenn kein anderer Entwickler die Datei bereits ausgecheckt hat.

Aufgabe 5)
Erläutern Sie kurz (2-3 Sätze): Was versteht man in der Informatik unter Technische Schulden?
Wofür kann man den Begriff in der Kommunikation mit Stakeholdern verwenden?

- Technische Schulden: Sie bezeichnen den zusätzlichen Aufwand, den man für zukünftige Änderungen und Erweiterungen an schlecht geschriebener Software im Vergleich zu gut geschriebener Software einplanen muss.
- Kommunikation mit Stackholdern: Der Begriff wird genutzt, um Managern und Stakeholdern klarzumachen, dass das Aufschieben von qualitätssichernden Maßnahmen die Softwareentwicklung nicht beschleunigt, sondern diese auf lange Sicht verlangsamt.

Aufgabe 6)
Erläutern Sie kurz (2-3 Sätze): Was versteht man im Zusammenhang mit Versionsverwaltung unter Merge?

→ Ein Merge (Dateiverschmelzung) ist das Integrieren von bereits im Repository vorhandenen Änderungen in die eigene, lokale Datei. Dieser Schritt ist beim Einchecken notwendig, wenn eine Datei von mehreren Entwicklern gleichzeitig ausgecheckt und in der Zwischenzeit bereits von einem anderen Entwickler aktualisiert wurde. Die Zusammenführung dieser beiden Dateiversionen kann dabei entweder völlig konfliktfrei erfolgen oder zu einem Konflikt führen.

Zusammenfassung der SOLID-Prinzipien

Die SOLID-Prinzipien sind konkrete Programmierwerkzeuge, um Code wartbarer, fehlerresistenter und anpassungsfähiger zu machen.

- **Single Responsibility Principle (SRP):** Eine Klasse darf nur genau eine einzige Verantwortung haben. Beispielsweise sollten reine Datenklassen (wie `Person`, `Kurs` oder `Mitarbeiter`) von Speicherlogik (Datenbank) oder anderen Domänen (wie `Buero` oder `Volkshochschule`) getrennt sein. Das reduziert Seiteneffekte, da es weniger Gründe gibt, eine Klasse ändern zu müssen.

- **Open Closed Principle (OCP):** Softwarekomponenten sollen offen für Erweiterungen, aber geschlossen für Modifikationen sein. Das bedeutet, dass neue Features (z. B. neue Holzarten bei der Preisberechnung) hinzugefügt werden können, ohne bestehenden Code verändern zu müssen. Dies senkt die Fehleranfälligkeit bei der Wartung.

- **Liskov Substitution Principle (LSP):** Unterklassen müssen anstelle ihrer Oberklassen einsetzbar sein, ohne dass es zu unerwartetem Verhalten kommt. Überschriebene Methoden (wie `gibAttributeAus()` bei Teppichen oder Salben, oder `toString()` bei Sportvereinen) dürfen den Aufrufer nicht überraschen. Dies garantiert, dass polymorphe Aufrufe sicher und korrekt ausgeführt werden.

- **Interface Segregation Principle (ISP):** Große Schnittstellen müssen in kleinere, spezifischere aufgeteilt werden (z.B. die Trennung von allgemeiner `Kreisberechnung` und spezifischer `Volumenberechnung`). Keine Klasse (wie ein einfacher `Bahnhof` oder `Kreis`) darf gezwungen werden, Methoden zu implementieren, die sie gar nicht benötigt (wie Gebäudefunktionen oder Volumen). Das führt zu einer geringeren Kopplung.

- **Dependency Inversion Principle (DIP):** Module höherer Ebenen (High-Level, z.B. das `Anwendungssystem`) dürfen nicht von Modulen niedrigerer Ebenen (Low-Level, z.B. `Kreis` oder `Kugel`) abhängen. Beide sollten von Abstraktionen (Interfaces wie `Berechnungen`) abhängen.

Clean Code

Während SOLID das "Wie" (die Werkzeuge) beschreibt, definiert Clean Code das "Warum" (die Ziele) anhand von Werten und Prinzipien, die in verschiedene Grade (Lernstufen, z.B. Roter Grad) eingeteilt sind.

- **Das Wertesystem:**
  - **Wandelbarkeit:** Die Kosten für neue Features dürfen nicht von der Menge des bestehenden Codes abhängen. Neue Anforderungen (z.B. Cloud-Speicher) müssen leicht integrierbar sein.
  - **Korrektheit:** Der Code muss fehlerfrei sein, was oft durch systematische JUnit-Tests und zustandsbasierte Tests gesichert wird.
  - **Produktionseffizienz & Kontinuierliche Verbesserung:** Software soll kostengünstig erstellt werden, und Teams sollen sich stetig weiterentwickeln.

- **Prinzipien & Praktiken (Roter Grad):**
  - **DRY (Don't Repeat Yourself):** Jede Code-Duplizierung führt zu Inkonsistenzen und Fehlern und muss vermieden werden.
  - **IOSP (Integration Operation Segregation Principle):** Integration (Koordination/Verwaltung) und Operation (Berechnungen/Logik) müssen in getrennten Methoden liegen.
  - **Favour Composition over Inheritance (FCoI):** Komposition (das Einbinden von Objekten als Attribute, z.B. ein Gebäude-Attribut in `Bahnhof`) ist der Vererbung vorzuziehen.
  - **Tue nur das Nötigste:** Vorsicht vor verfrühten Optimierungen; zusätzlicher Aufwand für Dinge, die nicht gefordert sind, ist Zeitverschwendung.

- **Refaktorisierung:** Werkzeuge wie *Extract Method* oder *Chain Constructor* (Konstruktoren, die sich über `this(...)` gegenseitig aufrufen) helfen direkt dabei, diese Clean-Code-Prinzipien (wie DRY) in der Praxis umzusetzen.